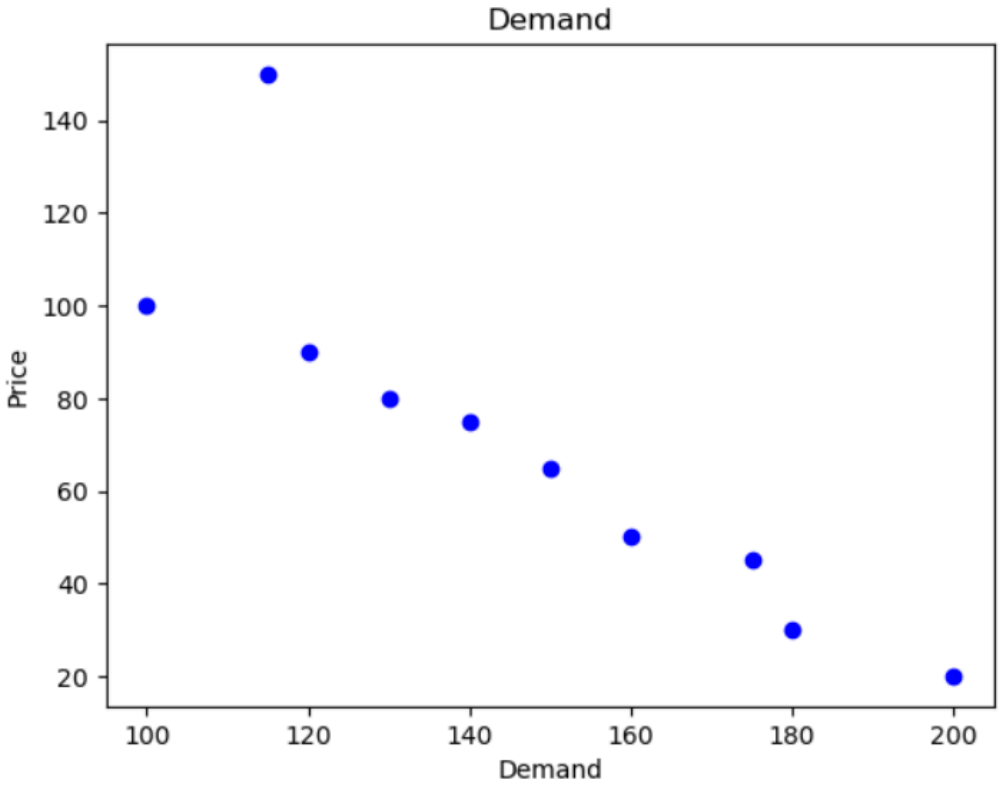
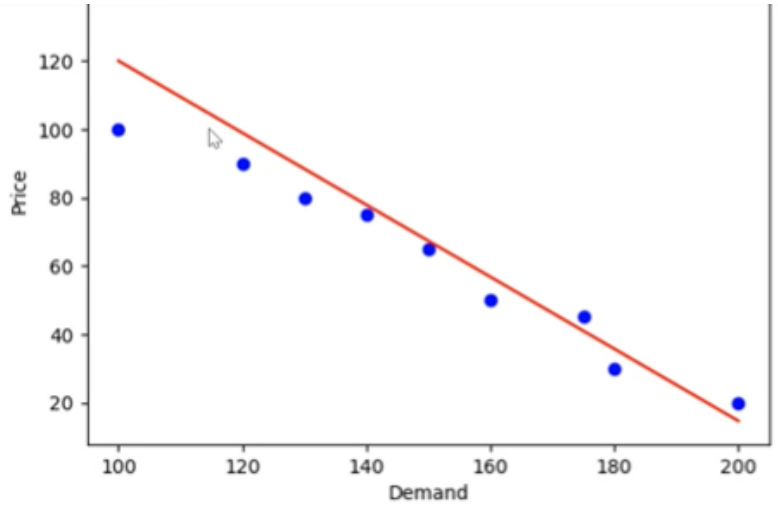
```
# Scatter plot
price = [20, 30, 45, 50, 65, 75, 80, 90, 150, 100]
demand = [200, 180, 175, 160, 150, 140, 130, 120, 115, 100]
plt.scatter(demand, price, colour = 'blue')
plt.title('Demand')
plt.xlabel('Demand')
plt.ylabel('price')
plt.show()
```



```python
# Scatter plot and add regression line (y=mx+b)
import numpy as np
price = [20,30, 45, 50, 65, 75, 80, 90, 150, 100]
demand = [200, 180, 175, 160, 150, 140, 130, 120, 115, 100]
m,b = np.polyfit(demand,price,1)
# create line value accross the range
x_line = np.linespace(min(demand), max(demand), 100)
y_line = m*x_line+b
#plot regression line
plt.plot(x_line, y_line, color = 'red', label = f'line:y={m:.2f}x+ {b:.2f}')
plt.legend()
plt.scatter(demand, price, color = 'blue')
plt.title('demand')
plt.xlabel('demand')
plt.ylabel('price')
plt.show()
```




```python
#OOP BASIC CLASS TEMPLATE
class className:
    #constructor
    def __init__(self, param1, param2):
        self.param1 = param1
        self.param2 = param2
    def instant_method(self):
        return f'param1 is {self.param1}'
obj = className('Hello', 123)
print(obj.instant_method())
```

```
param1 is Hello
```

```python
# class that returns product details
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price
    def label(self):
        return f' {self.name}- R{self.price}'
p = Product('Computer HP', 2500)
print(p.label())
```

```
 Computer HP- R2500
```

```python
#calculate total price  (name, price, quauntity bought)
class Product:
    def __init__(self, name, price, quantity):
        self.name = name
        self.price = price
        self.quantity = quantity
    def total_price(self):
        return self.price * self.quantity
    def display(self):
        return f'{self.name} total price is {self.total_price()}'
#create object
item = Product('Apples', 10, 3)
print(item.display())
```

```
Apples total price is 30
```

```python
#class BankAccount default parameter for the balance owner, balance
#deposit amount
class BankAccount:
    def __init__(self, owner, balance=0):    #default value
        self.owner = owner
        self.balance = balance
    def deposit(self, amount):
        self.balance += amount
        return f'{amount} deposited. New balance is {self.balance}'
#create object
acc1 = BankAccount('Fatima')   #balance defaults to 0
print(acc1.deposit(500))
```

```
500 deposited. New balance is 500
```

```python
class Student:  # instant variables
    def __init__(self,name, score):
        self.name = name
        self.score = score
s1 = Student('Alice', 80)
s2 = Student('Peter', 90)
s3 = Student('Bob', 20)
print(s1.name, s1.score)
```

```
Alice 80
```

```python
#class variable
class Student:
    school = 'DUT'
    def __init__(self, name):
        self.name = name
s1 = Student('Alice')
s2 = Student('Bob')
print(s1.school)
print(s1.school)
print(s1.name)
print(s2.name)
```

```
DUT
DUT
Alice
Bob
```